OPERATING SYSTEMS – ASSIGNMENT 2

CO2: Process management, scheduling, threads, IPC, synchronisation and deadlocks

Name: _____ Register Number: _____

Course: CSA04 – Operating Systems

Note: The questions below are restated from the supplied assignment sheet, followed by model answers. Replace measured results and execution screenshots with your own Linux/GCC results before submission.

Q1 – Processes and Process States

ASSIGNMENT QUESTION

Draw and explain the process-state diagram (new, ready, running, waiting, terminated) and PCB. Write/describe a fork() program and observe processes using ps and /proc. Evaluate 2,000 active processes and choose among process-per-student, thread-per-student and a worker pool. Compare sessions as threads.

AIM

To understand processes and process states and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

Process states:

TOPIC	EXPLANATION
New	process is being created.
Ready	process is in memory and waiting for CPU time.
Running	instructions are executing.
Waiting/blocked	process waits for I/O or an event.
Terminated	execution has finished.
Transitions	New → Ready when admitted; Ready → Running when scheduled; Running → Ready on pre-emption; Running → Waiting when requesting

	I/O; Waiting → Ready when the event completes; Running → Terminated on exit.

PCB contents include PID, process state, program counter, CPU registers, scheduling priority and queue pointers, memory-management information, open-file descriptors, accounting information and I/O status.

Example observation program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t p = fork();
    if (p == 0) { printf("child pid=%d parent=%d\n", getpid(), getppid()); _exit(0);
}
    if (p > 0) { printf("parent pid=%d child=%d\n", getpid(), p); waitpid(p, NULL,
0); }
    return 0;
}
```

For 2,000 sessions, one process per student causes high address-space and context-switch overhead. One thread per student reduces memory but shares failure domains. A fixed worker pool provides bounded resource use and queueing control, so it is the recommended design. Use `ps -o pid,ppid,state,comm` and inspect `/proc/<pid>/status` and `/proc/<pid>/maps`.

Q2 – CPU Scheduling

ASSIGNMENT QUESTION

Using jobs P1–P4 with arrival times 0,1,2,3 and burst times 8,4,9,5, calculate Gantt charts, waiting, turnaround and response times for FCFS, SJF, SRTF, priority and Round Robin. Implement Round Robin and evaluate quantum 1,3,5,20, starvation and MLFQ.

AIM

To understand cpu scheduling and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

For the supplied jobs, use non-preemptive SJF and priority only after defining priorities; the assignment does not provide priority values, so priority scheduling results cannot be

uniquely calculated. The following values use FCFS, non-preemptive SJF, SRTF and Round Robin (q=3). TAT = completion − arrival; WT = TAT − burst; response = first start − arrival.

TOPIC	EXPLANATION
FCFS Gantt	P1 0–8, P2 8–12, P3 12–21, P4 21–26.

Completion/TAT/WT/response:

TOPIC	EXPLANATION
P1	8/8/0/0; P2: 12/11/7/7; P3: 21/19/10/10; P4: 26/23/18/18.

Average TAT=15.25; WT=8.75; response=8.75.

TOPIC	EXPLANATION
Non-preemptive SJF	P1 0–8, P2 8–12, P4 12–17, P3 17–26.
P1	TAT 8, WT 0, response 0; P2: 11,7,7; P4: 14,9,9; P3: 24,15,15.

Average TAT=14.25; WT=7.75; response=7.75.

TOPIC	EXPLANATION
SRTF	P1 0–1, P2 1–5, P1 5–12, P4 12–17, P3 17–26.
P1	TAT 12, WT 4, response 0; P2: 4,0,0; P3: 24,15,15; P4: 14,9,9.

Average TAT=13.50; WT=7.00; response=6.00.

Round Robin q=3 (matching the supplied program's queue convention): Gantt: P1 0–3, P2 3–6, P3 6–9, P4 9–12, P1 12–15, P2 15–16, P3 16–19, P4 19–21, P1 21–23, P3 23–26.

TOPIC	EXPLANATION
Completion	P1=23, P2=16, P3=26, P4=21.

TAT	23,15,24,18. WT: 15,11,15,13. Response: 0,2,4,6.

Average TAT=20.00; WT=13.50; response=3.00.

Quantum 1 improves responsiveness but increases context switches. Quantum 20 approaches FCFS and reduces switching but delays short jobs behind long jobs. SJF/SRTF can starve long jobs; priority scheduling can starve low-priority jobs. Ageing gradually increases waiting jobs' priority. MLFQ starts new/interactive jobs in short-quantum queues and moves CPU-heavy jobs to longer-quantum queues, balancing response and throughput.

Round Robin TODO core:

```
int run = left[p] < quantum ? left[p] : quantum;
int old = time;
time += run;
left[p] -= run;
printf(" %s(%d-%d)", name[p], old, time);
for (int i = 0; i < N; i++)
    if (!added[i] && arrival[i] <= time) { queue[tail++] = i; added[i] = 1; }
if (left[p] > 0) queue[tail++] = p;
else { finish[p] = time; done++; }
```

The supplied expected output's ordering reflects its queue insertion convention. Verify the actual implementation against the supplied code and record any differences caused by when newly arrived jobs are enqueued.

Q3 – Threads

ASSIGNMENT QUESTION

Process frames from 64 cameras using one thread per camera and pools of 4, 8 and 16. Explain shared/private resources, thread models, speed-up limits and race conditions. Compare threads with processes.

AIM

To understand threads and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

Threads in one process share code, global data, heap, open files and address space. Each thread has its own program counter, registers, stack and thread ID. User-level threads are

managed by a runtime; kernel-level threads are scheduled by the OS. Many-to-one maps many user threads to one kernel thread; one-to-one maps each user thread to a kernel thread; many-to-many multiplexes user threads over a set of kernel threads.

A pool avoids creating 64 threads for every batch and limits simultaneous work. Measure elapsed time for sequential, 4, 8 and 16 workers. Speed-up is limited by CPU core count, I/O, lock contention, memory bandwidth and serial work (Amdahl's law). Unsynchronised updates to a shared counter can lose increments; use a mutex or atomic operation.

Threads usually have cheaper creation/switching and share data directly. Processes offer stronger fault isolation but need IPC and separate address-space resources. For this camera service, a bounded thread pool is a good choice when processing libraries are thread-safe; isolate untrusted or crash-prone decoders in worker processes.

Q4 – Inter-Process Communication

ASSIGNMENT QUESTION

Implement sensor-reader/analyser exchange using a pipe and shared memory or message queue. Explain EOF signalling, copying and synchronisation; measure transfer sizes 1 MB, 10 MB and 100 MB. Complete the pipe TODO and compare shared memory.

AIM

To understand inter-process communication and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

A pipe is a byte stream between processes. The reader sees EOF only after every open write end has been closed. Therefore the parent must close fd[1]; otherwise the child continues waiting for more data. A pipe copies data through kernel-managed buffers and provides synchronisation through blocking reads/writes. Shared memory allows both processes to access the same mapped region without copying each payload through the kernel, but requires explicit synchronisation, for example semaphores or process-shared mutexes. Message queues preserve message boundaries but add kernel copying and queue management.

Completed child TODO:

```
while (read(fd[0], &r, sizeof r) == sizeof r) {
   if (count == 0 || r > max) max = r;
```

```
    sum += r;
    count++;
}
```

TOPIC	EXPLANATION
Expected output	analyser: 6 readings, average 35.5, max 41; reader: done. For a pipe, transfer time grows approximately O(n) with bytes transferred. Shared memory also requires O(n) data access but avoids repeated kernel copies; setup and synchronisation costs can dominate small transfers.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors

ASSIGNMENT QUESTION

Explain mutual exclusion, progress and bounded waiting; Peterson's algorithm; binary/counting semaphores; bounded-buffer, readers-writers and dining-philosophers; monitors and condition variables. Test the bounded buffer and compare lock granularities and atomics/spinlocks.

AIM

To understand synchronisation: critical sections, semaphores and monitors and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

A critical-section solution must ensure mutual exclusion (only one process enters at a time), progress (selection of the next entrant cannot be postponed indefinitely when the section is free) and bounded waiting (a process has a finite bound on overtaking).

Peterson's two-process algorithm uses intent flags and a turn variable to coordinate entry. It demonstrates the principles but relies on memory-order assumptions; modern CPUs and compilers may reorder operations, so proper atomic operations and memory barriers are needed.

A binary semaphore has values 0/1 and protects one resource; a counting semaphore tracks multiple identical resources. wait() decrements or blocks; signal() increments and

wakes a waiter. In bounded-buffer, `empty` counts free slots, `full` counts occupied slots, and a mutex protects indices/data. Readers-writers uses reader counts and locks to coordinate readers and writers. Dining philosophers requires a deadlock-safe resource ordering or a waiter/semaphore limiting simultaneous acquisition.

A monitor encapsulates shared data and procedures; condition variables allow threads to wait and be signalled while maintaining mutual exclusion. It is easier to reason about than scattered semaphore operations.

Completed producer/consumer TODOs:

```
/* producer */
sem_wait(&empty);
pthread_mutex_lock(&lock);
buffer[in] = item;
in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&full);

/* consumer */
sem_wait(&full);
pthread_mutex_lock(&lock);
int item = buffer[out];
out = (out + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&empty);
__atomic_fetch_add(&consumed_sum, item, __ATOMIC_RELAXED);
```

For multiple producers/consumers, distribute unique item ranges and make the shared sum atomic or protect it with the mutex. One global lock is simplest but limits throughput; per-train locks improve concurrency while preserving seat consistency; per-coach locks provide finer concurrency but require careful coordination for cross-coach operations. Spinlocks waste CPU while waiting and suit only very short critical sections; blocking mutexes/semaphores suit longer waits.

Q6 – Deadlock Handling and Banker's Algorithm

ASSIGNMENT QUESTION

State the four necessary deadlock conditions and show them in a resource-allocation graph. Complete the Banker safety/request algorithms; evaluate prevention, avoidance, detection/recovery and ignoring deadlock; calculate safe sequence and requests; predict complexity and recommend a strategy.

AIM

To understand deadlock handling and banker's algorithm and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

The four necessary conditions are mutual exclusion, hold-and-wait, no pre-emption and circular wait. A resource-allocation graph uses process nodes and resource nodes; request edges point process→resource and assignment edges resource→process. A cycle is necessary for deadlock in single-instance resource types and indicates possible deadlock for multiple instances.

Banker's algorithm calculates Need = Max − Allocation. Set Work = Available and Finish=false. Repeatedly find an unfinished process whose Need is ≤ Work; pretend it finishes, add its Allocation to Work, mark it finished and append it to the safe sequence. If all processes finish, the state is safe. For a request, first ensure Request ≤ Need and Request ≤ Available; tentatively allocate, run safety test, and keep the allocation only if safe.

TOPIC	EXPLANATION
Given state safe sequence	P1, P3, P4, P0, P2.
P1 request (1,0,2)	granted; safe sequence remains P1 P3 P4 P0 P2.
P4 request (3,3,0)	must wait because resources are not currently available.
P0 request (0,2,0)	tentative allocation makes the state unsafe, so it is denied and rolled back.
For P0, before request	Allocation=(0,1,0), Max=(7,5,3), Need=(7,4,3). After tentative request, Allocation=(0,3,0), Need=(7,2,3), Available becomes (2,1,2). No unfinished process can complete with this Work vector, so no safe sequence exists.

Prevention removes at least one necessary condition, but can reduce utilisation. Avoidance with Banker's algorithm grants only safe requests but requires maximum demands in advance. Detection/recovery allows allocation and periodically searches for cycles, but recovery may abort jobs and lose work. Ignoring deadlock is acceptable only when deadlocks are rare and recovery cost is low; it is unsuitable for critical shared laboratory resources.

TOPIC	EXPLANATION
Complexity	for n processes and m resource types, the safety test is $O(m\ n^2)$ in the straightforward implementation, with O(mn) storage for matrices plus O(m+n) working arrays. Measure 10, 100 and 1,000 processes on the same machine; times are environment-dependent and should be reported from actual runs. For a research cluster with known job maxima, Banker's avoidance is appropriate for scarce non-preemptible resources; combine it with timeouts and operator recovery for unknown workloads.

```
for (int r = 0; r < R; r++) {
   int need = maxm[p][r] - alloc[p][r];
   if (need > work[r]) { found = 0; break; }
   if (r == R - 1) found = 1;
}
if (found) {
   for (int r = 0; r < R; r++) work[r] += alloc[p][r];
   finish[p] = 1;
   seq[count++] = p;
}
```

TOPIC	EXPLANATION
Implementation note	initialise found=1 before checking all resources, and set it to 0 if any resource need exceeds Work. Continue scanning until a complete safe sequence is found or no unfinished process can proceed.